

Lab05_Blumtritt

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.2
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(lubridate)
library(leaflet)
library(scales)
```

Attaching package: 'scales'

The following object is masked from 'package:purrr':

discard

The following object is masked from 'package:readr':

col_factor

```

#leitura em chunks filtrando por TAIL_NUMBER
analisa_aeronave <- function(tail_number, arquivo, arquivo_airports = "airports.csv") {
  callback_filtro <- readr::DataFrameCallback$new(function(chunk, pos) {
    dplyr::filter(chunk, TAIL_NUMBER == tail_number)
  })
  dados_aeronave <- readr::read_csv_chunked(
    file = arquivo,
    callback = callback_filtro,
    chunk_size = 100000,
    show_col_types = FALSE
  )
  if (nrow(dados_aeronave) == 0) {
    stop("Nenhum registro para essa aeronave")
  }
}

#leitura dos aeroportos e renomeação das colunas selecionadas
dados_aeroportos <- readr::read_csv(arquivo_airports, show_col_types = FALSE) %>%
  select(
    code = IATA_CODE,
    lat = LATITUDE,
    lon = LONGITUDE,
    nome = AIRPORT
  )

#processamento da tabela
tabela_processada <- dados_aeronave %>%
  filter(!is.na(DEPARTURE_TIME), !is.na(AIR_TIME)) %>%
  mutate(
    horario_partida_str = sprintf("%04d", as.integer(DEPARTURE_TIME)),
    hora_partida = as.numeric(substr(horario_partida_str, 1, 2)),
    minuto_partida = as.numeric(substr(horario_partida_str, 3, 4)),
    data_hora = lubridate::make_datetime(YEAR, MONTH, DAY, hora_partida, minuto_partida),
    velocidade_media_mph = (DISTANCE / AIR_TIME) * 60) %>%
  arrange(data_hora) %>%
  left_join(dados_aeroportos, by = c("ORIGIN_AIRPORT" = "code")) %>%
  rename(lat_origem = lat, lon_origem = lon, nome_origem = nome) %>%
  left_join(dados_aeroportos, by = c("DESTINATION_AIRPORT" = "code")) %>%
  rename(lat_destino = lat, lon_destino = lon, nome_destino = nome) %>%
  filter(!is.na(lat_origem), !is.na(lat_destino))

#construindo o mapa
mapa_trajeto <- leaflet() %>%

```

```

    addTiles()
  for (i in seq_len(nrow(tabela_processada))) {
    voo_atual <- tabela_processada[i, ]
    espessura_linha <- scales::rescale(
      voo_atual$velocidade_media_mph,
      to = c(2, 8),
      from = c(100, 600)
    )
    mapa_trajeto <- mapa_trajeto %>%
      addPolylines(
        lng = c(voo_atual$lon_origem, voo_atual$lon_destino),
        lat = c(voo_atual$lat_origem, voo_atual$lat_destino),
        weight = ifelse(is.na(espessura_linha), 2, espessura_linha),
        color = "red",
        opacity = 0.6,
        popup = paste0(
          "<b>Voo: </b>", voo_atual$AIRLINE, " ", voo_atual$FLIGHT_NUMBER, "<br>",
          "<b>Data/Hora: </b>", format(voo_atual$data_hora, "%d/%m/%Y %H:%M"), "<br>",
          "<b>Trecho: </b>", voo_atual$ORIGIN_AIRPORT, " ", voo_atual$DESTINATION_AIRPORT,
          "<b>Velocidade Média: </b>", round(voo_atual$velocidade_media_mph, 1), " mph"
        )
      )
  }
  aeroportos_unicos <- bind_rows(
    tabela_processada %>% select(IATA = ORIGIN_AIRPORT, lat = lat_origem, lon = lon_origem, nome = nome),
    tabela_processada %>% select(IATA = DESTINATION_AIRPORT, lat = lat_destino, lon = lon_destino, nome = nome)
  ) %>% distinct(IATA, .keep_all = TRUE)
  mapa_trajeto <- mapa_trajeto %>%
    addCircleMarkers(
      data = aeroportos_unicos,
      lng = ~lon,
      lat = ~lat,
      radius = 4,
      color = "blue",
      fillOpacity = 0.9,
      popup = ~paste0("<b>", nome, "</b> (", IATA, ")")
    )
  return(list(
    tabela = tabela_processada,
    mapa = mapa_trajeto
  ))
}

```

O chunk abaixo é o responsável por gerar o mapa e a tabela, porém, para conseguir renderizar, pelo código conter `readline()`, que espera uma entrada, o chunk não é rodado.

```
tail_input <- readline(prompt = "Digite o TAIL_NUMBER da aeronave (ex: N431WN): ")

resultado <- analisa_aeronave(
  tail_number = trimws(tail_input),
  arquivo = file.path("flights.csv.zip"),
  arquivo_airports = file.path("airports.csv")
)

#ver a tabela em ordem
head(resultado$tabela)

#mapa interativo dos vôos
resultado$mapa
```

Exemplo de chamada da função

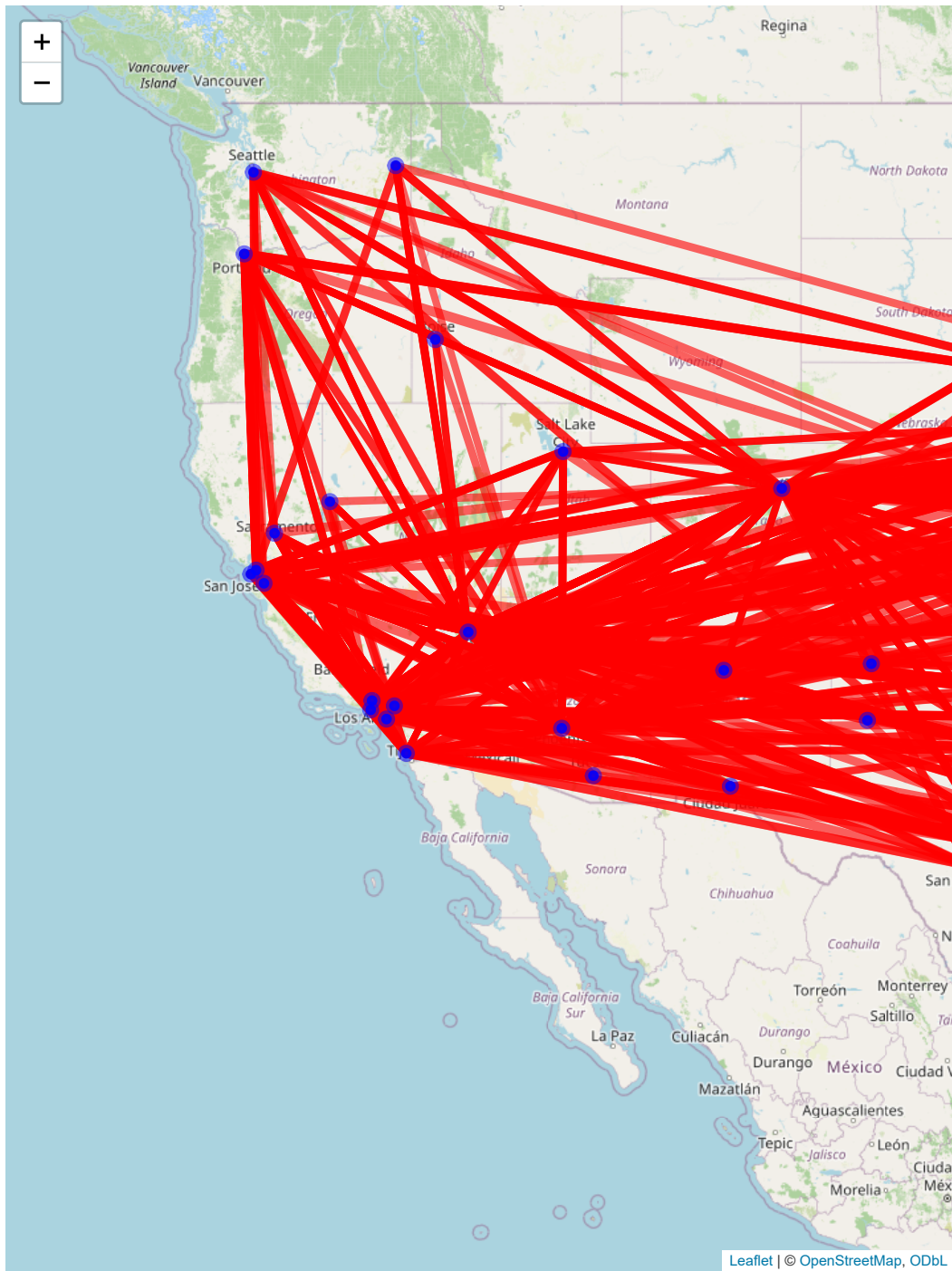
```
resultado <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo = file.path("flights.csv.zip"),
  arquivo_airports = file.path("airports.csv")
)

#ver a tabela em ordem
head(resultado$tabela)
```

```
# A tibble: 6 x 42
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER ORIGIN_AIRPORT
  <dbl> <dbl> <dbl>      <dbl> <chr>          <dbl> <chr>      <chr>
1  2015     1     1          4 WN             1762 N431WN     DCA
2  2015     1     1          4 WN             3564 N431WN     HOU
3  2015     1     1          4 WN             3564 N431WN     BOS
4  2015     1     1          4 WN             3564 N431WN     BWI
5  2015     1     2          5 WN             1793 N431WN     MCI
6  2015     1     2          5 WN             1793 N431WN     LAX
# i 34 more variables: DESTINATION_AIRPORT <chr>, SCHEDULED_DEPARTURE <chr>,
# DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>, TAXI_OUT <dbl>,
# WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>, AIR_TIME <dbl>,
# DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>, SCHEDULED_ARRIVAL <chr>,
# ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>, DIVERTED <dbl>, CANCELLED <dbl>,
```

```
# CANCELLATION_REASON <chr>, AIR_SYSTEM_DELAY <dbl>, SECURITY_DELAY <dbl>,  
# AIRLINE_DELAY <dbl>, LATE_AIRCRAFT_DELAY <dbl>, WEATHER_DELAY <dbl>, ...
```

```
#mapa interativo dos vôos  
resultado$mapa
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 17:06:37 -03"
```